MODULE 4

Swift Programming Fundamentals

Enterprise Mobile Application Development Bootcamp | AI-Integrated Edition

Module 4 at a Glance

1

The Swift Mental Model

2

Variables, Types & Flow

3

Functions & Closures

4

OOP in Swift

5

Swift Safety Features

Lab

Banking Project

©Logical Imagination Group LLC

Learning Objectives

- Explain why Swift is statically typed and the advantages over Python and JavaScript
- Declare variables and constants and choose correctly between let and var
- Write control flow: if/else, exhaustive switch, guard, and all loop types
- Define functions using labeled parameters, default values, and tuple returns
- Write closures and use map, filter, and reduce on collections
- Model domain concepts using classes, structs, enumerations, and protocols
- Explain Automatic Reference Counting and identify and fix retain cycles
- Safely unwrap optionals using if let, guard let, and optional chaining
- Write throwing functions and handle typed errors with do-catch
- Write generic functions and types constrained by protocol requirements

©Logical Imagination Group LLC

The Swift Mental Model

- Why Swift exists
- Static vs. Dynamic typing
- Safety as a design principle

©Logical Imagination Group LLC

Why Swift Exists

Type Safety

The compiler knows the type of every value at build time.
Type mismatches are build errors - not crashes in production.

Optionality is Explicit

A type either can or cannot be nil - and the contract is part of the type system, not a comment or convention.

Immutability by Default

let creates a true constant. The compiler prevents reassignment - no runtime check needed.

Errors Are Typed

Throwing functions declare that they can fail. Callers must handle every failure mode - the compiler enforces this.

©Logical Imagination Group LLC

Python/JS vs. Swift - The Core Differences

Python / JavaScript

x = 5; x = 'hello' # works

```
x = 5
x = 'hello'  # no error
```

None can appear anywhere (convention, not enforcement)

```
user = None  # always valid
```

Swift

Static typing: type is fixed at declaration

```
var x = 5
x = "hello"  // compile error
```

Optional<T> - nil must be declared

```
var user: String?  // opt-in
var name: String   // never nil
```

©Logical Imagination Group LLC

Python/JS vs. Swift - Errors and Mutability

Python / JavaScript

Variable can be anything

```
x = 5
x = 10  # fine, no warning
```

raise ValueError() - optional to catch

```
def login():  raise ValueError()
login()  # no catch required
```

Swift

let = truly immutable constant

```
let x = 5
x = 10  // compile error
```

throws - caller MUST handle it

```
func login() throws -> Token
try login()  // must be in do-catch
```

©Logical Imagination Group LLC

Review

Skill Check ✓

- In one sentence: what does 'statically typed' mean?
- If a Swift variable is a non-optional String - what values can it hold?
- Name one type of bug that Swift would catch at compile time that Python would only catch at runtime.
- Why might a compiler error be BETTER than a runtime error?

©Logical Imagination Group LLC

Variables, Types & Control Flow

- let vs. var
- Type inference
- Strings
- Type conversion
- if/else
- switch
- guard
- Loops

©Logical Imagination Group LLC

let vs. var - Mutability is a Choice

- let - constant, cannot be reassigned
- Reach for let FIRST
- Switch to var only when mutation is required
- Compiler enforces this - cannot be forgotten
- var - variable, can be reassigned
- Use for values that must change
- Mutability communicates intent to readers

Code Bite

```
let appName = "PNC Mobile"  // constant
var loginAttempts = 0       // variable

loginAttempts += 1          // allowed
appName = "Other"           // COMPILE ERROR

// Type inference - no annotation needed
let balance = 4_250.75   // inferred: Double
// Explicit annotation
var interestRate: Double = 0.035
// _ as thousands separator (readability)
let limit = 5_000_000
```

©Logical Imagination Group LLC

Type Safety and Explicit Conversion

```
// No implicit numeric conversion
let count: Int = 47
let total: Double = 12_309.88
// COMPILE ERROR: cannot convert Int to Double
let avg = total / count
// FIX: explicit conversion
let avg = total / Double(count)   // 261.91

// Optional conversion - safe, never crashes
let parsed = Int("2500")   // Int? - may be nil
let bad    = Int("abc")    // nil - no crash
if let amount = parsed {
    print("Amount: \(amount)")
} else {
    print("Invalid input")
}
```

Python does this

```
int('abc')
# → ValueError at RUNTIME

user says 'abc'
app crashes
```

Swift does this

```
Int("abc")
// → nil   (never crashes)

user says 'abc'
app handles gracefully
```

©Logical Imagination Group LLC

The Swift switch - Exhaustive Pattern Matching

- Exhaustive - must handle every case
  - Compiler error if you miss one
- Range patterns are possible
  - case 100...999: handles any Int in range
- Where clauses
  - case let x where x > 0 && x < 18:
- Tuple matching
- No fall-through by default
  - Use fallthrough keyword explicitly if needed

Code Bite

```
// Range matching - Python has no equivalent
switch creditScore {
case 800...850:  print("Exceptional")
case 740...799:  print("Very Good")
case 670...739:  print("Good")      // 714 lands here
case 580...669:  print("Fair")
default:         print("Poor")
// Remove default → COMPILE ERROR: not exhaustive
}
```

©Logical Imagination Group LLC

switch Enums - Must Be Exhaustive

- Exhaustive - must handle every case
  - Compiler error if you miss one
- case (.checking, true): ...
- No fall-through by default
  - Use fallthrough keyword explicitly if needed

Code Bite

```
// Enum switching - no default possible if exhaustive
switch transactionType {
case .credit:    processCredit()
case .debit:     processDebit()
case .transfer:  processTransfer()
case .fee:       processFee()
// No default needed - enum IS exhaustive
}
```

©Logical Imagination Group LLC

guard - Early Exit and Clean Preconditions

Without guard (pyramid of doom)

```
func process(amount: Double?,
             balance: Double) {
  if let amt = amount {
    if amt > 0 {
      if amt <= balance {
        // work buried 3 levels deep
        executeTransfer(amt)
      } else {
        print("Insufficient")
      }
    } else {
      print("Invalid amount")
    }
  } else {
    print("No amount")
  }
}
```

With guard (flat, readable)

```
func process(amount: Double?,
             balance: Double) {
  guard let amt = amount else {
    print("No amount")
    return   // exit here
  }
  guard amt > 0 else {
    print("Invalid amount")
    return   // exit here
  }
  guard amt <= balance else {
    print("Insufficient")
    return   // exit here
  }

  // work at LEFT MARGIN, always reached
  executeTransfer(amt)
}
```

©Logical Imagination Group LLC

Variables, Types, and Control Flow

Skill Check

- Find and follow the Swift code file in the GitHub repository for the Phase2/Module04/VariablesTypesFlow directory.

©Logical Imagination Group LLC

Review

Skill Check

- What error do you get if you add an Int and a Double without conversion? How do you fix it?
- You write a switch on an Int. You have cases for 1, 2, and 3. What must you add? What happens if you don't?
- Rewrite this in your head using guard: if let x = optional { if x > 0 { doWork() } }
- What is the difference between for-in with a where clause and calling .filter before iterating?

©Logical Imagination Group LLC

Functions & Closures

- Labeled parameters
- Default values
- Tuple returns
- Closures
- map/filter/reduce
- Capture lists

©Logical Imagination Group LLC

Labeled Parameters - Self-Documenting Call Sites

- By default, when calling a function, the parameter names must be used when passing values
- This makes the invocation more readable

Code Bite

```
func transfer(amount: Double,
  source: String,
  destination: String) -> Bool {
    print("\(source) → \(destination): $\(amount)")
    return true
}

// Call site reads like prose
transfer(amount: 500.00, source: "Checking",
  destination: "Savings")
```

©Logical Imagination Group LLC

Internal vs. External Names

- You can set up the function so that you can use a different parameter name when invoking the function
- When defining the function, list the external name first and then the internal name
  - Separate them with a space

Code Bite

```
func transfer(amount: Double,
  from source: String,
  to destination: String) -> Bool {
    // inside: use internal names
    print("\(source) → \(destination): $\(amount)")
    return true
}

// Call site uses external names
transfer(amount: 500.00, from: "Checking",
  to: "Savings")
```

©Logical Imagination Group LLC

Labeled Parameters - Suppressing Label Name

- Occasionally, you may want to invoke a function without providing the parameter name
- In this case, use an underscore ("_") as the external name

Code Bite

```
// Suppress first label with _
func authenticate(_ username: String,
  with password: String) -> Bool {
    return username == "jsmith"
      && password == "pass123"
}

authenticate("jsmith", with: "pass")
// clean call site
```

©Logical Imagination Group LLC

Parameter Default Values

- When defining a function, you can give any of its parameters a default value
- This makes the parameter optional

Code Bite

```
// Default values
func fetchTransactions(limit: Int = 50,
  offset: Int = 0) -> String { ... }

fetchTransactions()
// limit=50, offset=0

fetchTransactions(limit: 20)
// limit=20, offset=0

fetchTransactions(limit: 20, offset: 40)
// explicit both
```

©Logical Imagination Group LLC

Named Tuple Returns

- If you want to return more than one piece of data from a function, you can create a named tuple as the return type
- For the return type surround the multiple names/data types in parenthesis

Code Bite

```
func validate(amount: Double) ->
  (isValid: Bool, error: String?) {
    guard amount > 0 else {
      return (false, "Must be > 0")
    }
    return (true, nil)
}
let r = validate(amount: 500)
if r.isValid {
     proceed()
} else {
     show(r.error!)
}
```

©Logical Imagination Group LLC

Closure Syntax - From Full to Shorthand

1 **Full form**

```
numbers.sorted(by: { (a: Int, b: Int) -> Bool in
    return a < b
})
```

2 **Implicit return**

```
numbers.sorted(by: { (a: Int, b: Int) -> Bool in a < b })
```

3 **Inferred types**

```
numbers.sorted(by: { a, b in a < b })
```

4 **Trailing closure**

```
numbers.sorted { a, b in a < b }
```

5 **Shorthand arguments**

```
numbers.sorted { $0 < $1 }
```

©Logical Imagination Group LLC

Functional Collection Processing - iterating

- .forEach() will execute some behavior for each element in the array
- Void side effects only, there is no return

Code Bite

```
let balances = [3_250.00, 12_000.00, 450.75,
8_900.00, 125.50, 22_450.00]


balances.forEach( { (bal: Double) in
  print("The balance is $\(bal)")
} )

// or
balances.forEach {
  print("The balance is $\( $0 )")
}
```

©Logical Imagination Group LLC

Functional Collection Processing - filter

- .filter() will build a new array consisting of a subset of the original elements
- The closure should return a Boolean
  - Elements whose execution of the closure returns true will be included
- Returned is an array of the same type, with potentially fewer elements

Code Bite

```
let balances = [3_250.00, 12_000.00, 450.75,
8_900.00, 125.50, 22_450.00]


// filter - returns [T] of elements where
closure returns true

let highValue = balances.filter { $0 > 5_000
}

// [12000.0, 8900.0, 22450.0]
```

©Logical Imagination Group LLC

Functional Collection Processing - map

- You pass in a closure that translates one of the existing items from the array into the value that you wish included in the resultant array
- .map() will build a new array
  - It will contain the return values from all the invocations of your closure

Code Bite

```
let balances = [3_250.00, 12_000.00, 450.75,
8_900.00, 125.50, 22_450.00]


// map - transforms each element
let withInterest = balances.map {
  $0 * 1.035
}
// each balance increased by 3.5%
```

©Logical Imagination Group LLC

Functional Collection Processing - reduce

- .reduce() combines all the array values into one value
- Its parameters include the initial value for the summary and the closure you wish to use
- The closure accepts two parameters
  - The summary so far
  - The current value being combined
- The closure returns the modified summary
  - This value will be used on the next iteration
  - The last iteration's return will be the final return value from reduce()

Code Bite

```
let balances = [3_250.00, 12_000.00, 450.75,
8_900.00, 125.50, 22_450.00]


// reduce - collapses to a single value
let total = balances.reduce(0, +)
// 47176.25


let total2 = balances.reduce(0) { $0 + $1 }
// same thing
```

©Logical Imagination Group LLC

Chaining Calls

- Most of the array methods return a new array
- Calls can be chained together in a Fluent style

Code Bite

```
let balances = [3_250.00, 12_000.00, 450.75,
8_900.00, 125.50, 22_450.00]


// Chain - filter then map then reduce
let premiumTotal = balances
    .filter { $0 > 1_000 }
      // keep only large balances
    .map { $0 * 1.035 }
      // apply 3.5% interest
    .reduce(0, +)
      // sum everything
```

©Logical Imagination Group LLC

Functional Collection Processing - sorting

- .sorted() returns a new array with the same elements in a different order
- The closure you provide should return a Boolean
  - It will tell the algorithm whether the items being compared are in the right order or not

Code Bite

```
let balances = [3_250.00, 12_000.00, 450.75,
8_900.00, 125.50, 22_450.00]
```

©Logical Imagination Group LLC

Functions and Closures

Skill Check

- Find and follow the Swift code file in the GitHub repository for the Phase2/Module04/FunctionsClosures directory.

©Logical Imagination Group LLC

Review

Skill Check ✓

- What is the external label on a function parameter and what is the internal label? Give an example of each.
- What does $0 represent in a closure? What does $1 represent?
- You call .map { $0 * 2 } on [1, 2, 3]. What does it return? What type is it?
- Why would you use .reduce instead of a for loop with an accumulator variable?
- When should you add [weak self] to a closure capture list?

©Logical Imagination Group LLC

Object-Oriented Swift

- Structs (value types)
- Classes (reference types)
- Enumerations
- Protocol-Oriented Programming

©Logical Imagination Group LLC

struct Property Syntax

- struct is a value type
- Memberwise init is FREE - no need to write init()

Code Bite

```
struct Transaction {
    let id: String
    let date: Date
    let amount: Double
    var description: String
    let isDebit: Bool
}

let tx = Transaction(id: "001",
 date: Date(), amount: 500,
 description: "Deposit", isDebit: false)
```

©Logical Imagination Group LLC

struct Computed Properties

- Computed property - derived from stored data
- Define a function body after the variable declaration
- Property values can be accessed by directly working with the variables

Code Bite

```
struct Transaction {
  ...
  var formattedAmount: String {
    let prefix = isDebit ? "-" : "+"
    return "\(prefix)$\(String(
      format: "%.2f", amount))"
  }
}
let tx = Transaction(id: "001",
  date: Date(), amount: 500,
  description: "Deposit", isDebit: false)
print(tx.formattedAmount)   // +$500.00
```

©Logical Imagination Group LLC

struct Mutator Methods

- Any method that changes a struct property must have the mutating modifier on it
- The property that gets changed must be declared as a var and not a let constant

Code Bite

```
struct Transaction {
  ...
  mutating func addNote(_ note: String) {
    description = description + " (\(note))"
  }
}
```

©Logical Imagination Group LLC

struct is a Value Type

- Assigning one struct to a second variable results in independent copies
- This is not as inefficient as you may imaging
- Swift operates by a copy-on-write strategy, so a copy is not created until a property is changed

Code Bite

```
var t1 = Transaction(...)
var t2 = t1              // copy

t2.description = "Modified"

// t1 is UNCHANGED
print(t1.description)
// "Deposit" ← original

print(t2.description)
// "Modified" ← copy
```

©Logical Imagination Group LLC

class - Property Syntax

- Classes are reference types
- Multiple variables assigned the same object will be references to one instance

Code Bite

```
class BankAccount {
  let id: String
  let accountNumber: String
  var balance: Double
  let owner: String

  init(id: String, acctNum: String,
    bal: Double, owner: String) {
      self.id = id
      self.accountNumber = acctNum
      . . .
  }
}
```

©Logical Imagination Group LLC

class - Initialization and Deallocation

- Classes must have an initializer
  - Or else initialize all properties to some default value
- Automatic Reference Count (ARC) signal is the deinit() method
- Properties must be accessed through the self keyword

Code Bite

```
class BankAccount {

  ...
  init(id: String, acctNumber: String,
owner: String, balance: Double = 0.0) {
    self.id = id
    self.accountNumber = acctNumber
    self.balance = balance
    self.owner = owner
  }
  deinit {
    print("\(accountNumber) deallocated")
  }
}
```

©Logical Imagination Group LLC

class - Behavioral Methods

- Methods can access and/or modify property values
- Must use the self keyword to access them

Code Bite

```
class BankAccount {
  ...
  func deposit(amount: Double) {
    guard amount > 0 else { return }
    self.balance += amount
  }
  func withdraw(amount: Double) -> Bool {
    guard amount > 0, amount <= balance else
    { return false }
    self.balance -= amount
    return true
  }
}
```

©Logical Imagination Group LLC

class - Reference Semantics

- Assigning the value of one object reference to another variable creates a second reference to the same object instance
- Changes made to properties through one reference will be seen by the other reference(s)

Code Bite

```
let acc = BankAccount(balance: 1000)
let ref = acc   // SAME object

ref.deposit(amount: 500)

// BOTH show 1500 - same object
print(acc.balance)   // 1500.0
print(ref.balance)   // 1500.0
```

©Logical Imagination Group LLC

struct vs. class - Rule of Thumb

- Use struct for DATA that represents a VALUE:
  - Transaction, Address, UserProfile, Color.
- Use class for OBJECTS with IDENTITY and LIFECYCLE.

©Logical Imagination Group LLC

Enumerations - Typed, Exhaustive, Powerful

- Can have raw values of any type
  - e.g. to map to external API strings
- Exhaustive switch behavior
- Compiler error if any case is unhandled
- Adding a new case forces updates everywhere

Code Bite

```
enum AccountType: String {
  case checking   = "CHECKING"
  case savings    = "SAVINGS"
  case investment = "INVESTMENT"
  case credit     = "CREDIT"

  var displayName: String {
    switch self {
      case .checking: return "Checking Account"
      case .savings: return "Savings Account"
      case .investment: return "Investment Account"
      case .credit: return "Credit Card"
    }
  }
}
```

©Logical Imagination Group LLC

Enumeration Cases Can Have Different Typed Data

- Associated values - carry different data per case
- Associated values
- Each case carries different typed data
- Replaces dict/tuple anti-patterns

Code Bite

```
enum AccountError {
  case insufficientFunds(
    available: Double, requested: Double)
  case accountInactive
  case dailyLimitExceeded(limit: Double)
}
```

©Logical Imagination Group LLC

Enumerations - Typed, Exhaustive, Powerful

- Switch statements need to handle all possible values
- It is considered a best practice to NOT use default
  - This will force developers to visit each use of switch when a new enum value is created

Code Bite

```
switch error {
  case .insufficientFunds(let avail,
                          let req):
    print("Need $\(req), have $\(avail)")

  case .dailyLimitExceeded(let limit):
    print("Limit is $\(limit)")

  case .accountInactive:
    print("Account inactive")
}
```

©Logical Imagination Group LLC

Protocols Are Contracts

- Protocol defines a contract
- Structs AND classes can conform
- A type can adopt multiple protocols
- No coupling to a base class
- They are preferred in Swift

Code Bite

```
protocol Summarizable {
    var summary: String { get }  // required
    func printSummary()          // required
}
```

©Logical Imagination Group LLC

Extensions can Provide Implementation

- Extensions can add default behavior to an existing protocol
  - Conforming types will automatically acquire the behavior
- No conforming type needs to implement
  - They can, however, override the default behavior

Code Bite

```
// No conforming type needs to implement
extension Summarizable {
    func printSummary() { print(summary) }
}
```

©Logical Imagination Group LLC

Structs can Adopt Protocols

- Structs can conform to any protocol
- They must implement the properties defined in the protocol

Code Bite

```
struct Transaction: Summarizable {
  // ... properties ...
  var summary: String {
    "\(formattedDate)  \(description)
\(formattedAmount)"
  }

  // printSummary() inherited from
  // the extension
}
```

©Logical Imagination Group LLC

Classes can Adopt Protocols

- Classes can conform to any protocol
- They must implement the properties defined by the protocol

Code Bite

```
class BankAccount: Summarizable {
  // ... properties ...

  var summary: String { "\(displayName)
\(formattedBalance)" }

  // printSummary() inherited from
  // the extension

}
```

©Logical Imagination Group LLC

Protocols are Data Types

- The protocol data type can be used as a function parameter
- It can also be used as the return type from a function

Code Bite

```
func printAll(_ items: [Summarizable]) {
  items.forEach { $0.printSummary() }
}

let transaction1 = Transaction(. . .)
let transaction2 = Transaction(. . .)
let account = BankAccount(. . .)

printAll([transaction1,
  transaction2, account])
```

©Logical Imagination Group LLC

Protocols vs. Inheritance

- Protocol
  - Structs AND classes can conform
  - A type can adopt multiple protocols
  - No coupling to a base class
  - Extensions add default behavior
  - They are preferred over Inheritance
- Inheritance
  - Classes only
  - Single parent class
  - Tight coupling to base behavior
  - Use when you need override capacity

©Logical Imagination Group LLC

Object Oriented Swift

Skill Check

- Find and follow the Swift code file in the GitHub repository for the Phase2/Module04/ObjectOrientedSwift directory.

©Logical Imagination Group LLC

Review

Skill Check ✓

- You are designing a User type with a profile (name, email) and a session (token, expiry). Which should be a struct? Which should be a class? Why?
- You add a new case .wire to the TransactionType enum. What happens to the switch in AccountType.displayName?
- You call printSummary() on a BankAccount. BankAccount does not implement printSummary(). What happens?
- A protocol cannot be adopted by a struct if it requires... what? (trick question - think carefully)

©Logical Imagination Group LLC

Swift Safety Features

- ARC & retain cycles
- Optionals deep dive
- Typed error handling
- Generics

©Logical Imagination Group LLC

Reference Counting & Retain Cycles

- Customer has a strong reference to BankAccount (default)
- BankAccount has a strong reference to Customer (default)

Code Bite

```
class Customer {
    let name: String
    var account: BankAccount?
    init(name: String) { self.name = name }
    deinit { print("Customer \(name) dealloc") }
}
class BankAccount {
    let number: String
    var owner: Customer?
    init(number: String) { self.number = number }
    deinit { print("Account \(number) dealloc") }
}
```

©Logical Imagination Group LLC

Effect of Strong References

- Neither object is ever deallocated even when out of scope
- Strong refs increment the refcount for the objects
  - Customer ref to BankAccount keeps that object alive
  - BankAccount ref to Customer keeps that object alive

Code Bite

```
do {
   let jane = Customer(name: "Jane")
   let acc  = BankAccount(number: "ACC-001")
   jane.account = acc
   acc.owner    = jane
   . . .
}
// neither deinit fires - MEMORY LEAK
```

©Logical Imagination Group LLC

Fix #1 - Weak Ref

- Weak ref does not add to the refcount for the referenced object
- Automatically becomes nil when container object is deallocated
- Must be Optional and var
- Should use as a best practice for back-references
- Safe - nil check prevents crash
- Only one of the circular references needs to be weak

Code Bite

```
class Customer {
    let name: String
    var account: BankAccount?
    init(name: String) { self.name = name }
    deinit { print("Customer \(name) dealloc") }
}
class BankAccount {
    let number: String
    weak var owner: Customer?
    init(number: String) { self.number = number }
    deinit { print("Account \(number) dealloc") }
}
```

©Logical Imagination Group LLC

Effect of Weak References

- The BankAccount ref to Customer is set to nil when the BankAccount object is set to nil
- When the Customer variable is set to nil, there are no more references so its deinit fires
  - This removes the Customer ref to BankAccount which removes its last ref which allows BankAccount deinit to fire
- These objects are cleaned up immediately
  - No need to wait for a garbage collector to run

Code Bite

```
do {
   let jane = Customer(name: "Jane")
   let acc  = BankAccount(number: "ACC-001")
   jane.account = acc
   acc.owner    = jane
   . . .
}
// acc -> jane is automatically set to nil
// jane deinit fires, then acc deinit fires
```

©Logical Imagination Group LLC

Weak Reference in Closures

- The closure passed to the asynchronous operation (loadData) has a weak reference to the ViewController instance
- If ViewController instance is deallocated before the async operation completes the closure will exit
  - For example, if the user navigates away from the current view
  - The weak reference in the closure allows the refcount to reach zero

Code Bite

```
class ViewController: UIViewController {
  var dataLoader: DataLoader?

  func fetchData() {
    dataLoader?.loadData(completion: { [weak
self] result in
      guard let self = self else { return }

      switch result {
        case .success(let data):
          self.updateUI(with: data)
        case .failure(let error):
          self.showErrorMessage(error)
      }
    })
  }


}
```

©Logical Imagination Group LLC

Fix #2 - Unowned Ref

- Can work on constants and variables
- Does NOT become nil when dealloc
- Crashes if accessed after dealloc
- Use when lifetime is guaranteed
  - Customer may have a BankAccount
  - BankAccount ALWAYS has a Customer
  - BankAccount will not outlive the Customer

Code Bite

```
class Customer {
  let name: String
  var account: BankAccount?
  init(name: String) { self.name = name }
  deinit { print("Customer \(name) dealloc") }
}
class BankAccount {
  let number: String
  unowned let owner: Customer
  init(number: String, owner: Customer) {
    self.number = number
    self.owner = owner }
  deinit { print("Account \(number) dealloc") }
}
```

©Logical Imagination Group LLC

Effect of Unowned References

- Unowned ref from BankAccount to Customer does not contribute to refcount
- When the Customer variable is set to nil, there are no more references so its deinit fires
  - This removes the Customer ref to BankAccount which removes its last ref which allows BankAccount deinit to fire

Code Bite

```
do {
  let jane = Customer(name: "Jane")
  jane.account = BankAccount(number: "ACC-
001", jane)
  . . .
}
```

©Logical Imagination Group LLC

Optionals - Unwrapping Pattern #1

- The optional value, if not nil, is unwrapped into a new variable
- if-let creates a new scope for the new variable
- The new variable is no longer optional
- if-let allows for multi-binding
  - More than one let statement can be specified
  - The code block will only execute if all assignments are not nil

Code Bite

```
var username: String? = nil
username = "jsmith"

if let name = username {
  print("Hello, \(name)")
  // name is String, not String?
}
// name unavailable here
// Multi-binding - all or nothing
if let src = sourceId, let dst = destId, let
amt = amount, amt > 0 {
  transfer(src, dst, amt)
}
```

©Logical Imagination Group LLC

Optionals - Unwrapping Pattern #2

- guard-let provides an opportunity to exit a function if the optional is nil
- It also provides for creation of a new variable
  - This variable will hold the unwrapped value if the optional is not nil

Code Bite

```
var username: String? = nil

username = "jsmith"

func greet(_ username: String?) {
  guard let name = username else {
    print("No username")
    return
  }
  print("Hello, \(name)")
}
```

©Logical Imagination Group LLC

Optionals - Unwrapping Pattern #3

- nil coalescing provides the opportunity to specify a default value to be used if the optional is nil
- The new variable is unwrapped and never nil

Code Bite

```
var username: String? = nil

username = "jsmith"

let display = username ?? "Guest"
// display is String, not String?
```

©Logical Imagination Group LLC

Optionals - Unwrapping Pattern #4

- The safe access operator will only access the object property if the object reference is not nil
- The constant's value will itself be optional

Code Bite

```
var username: String? = nil

username = "jsmith"

let count = username?.count
// Int? - (nil if username nil)

let upper = username?.uppercased()
// String?
```

©Logical Imagination Group LLC

Optionals - Force Unwrapping

- Use force unwrap (!) ONLY when nil is a programmer error that should crash during development
- Safe: YOU wrote the string
  - URL(string: "https://api.pnc.com")!
- Safe: storyboard guarantees it
  - @IBOutlet weak var button: UIButton!
- UNSAFE - user input may fail
  - URL(string: userInput)!
  - Int(textField.text)!

Code Bite

```
let url = URL(string:
"https://api.pnc.com")!


@IBOutlet weak var button: UIButton!


let url = URL(string: userInput)!
// CRASH


let num = Int(textField.text)!
// CRASH
```

©Logical Imagination Group LLC

Creating Custom Error Types

- Define typed errors by creating an enum that conforms to Error or LocalizedError
- Functions can then throw these errors
- Calling code needs to handle them

Code Bite

```
enum TransferError: LocalizedError {
  case invalidAmount
  case insufficientFunds(available: Double)
  case dailyLimitExceeded(limit: Double,
attempted: Double)
  case networkUnavailable


  . . .
}
```

©Logical Imagination Group LLC

LocalizedError

- The errorDescription property is automatically used when the application requests the localizedDescription
- Override this method and provide a string value for each possible enum value

Code Bite

```
enum TransferError: LocalizedError {
  . . .
  var errorDescription: String? {
    switch self {
      case .invalidAmount:
        return "Amount must be greater than zero."
      case .insufficientFunds(let a):
        return "Available balance: $\(String(format:
"%.2f", a))"
      case .dailyLimitExceeded(let l, let a):
        return "Limit $\(l) exceeded. Attempted: $\(a)"
      case .networkUnavailable:
        return "Network unavailable. Try again."
    }
  }
}
```

©Logical Imagination Group LLC

Throwing Errors

- Functions can now throw instances of the error type
- These functions MUST be called with try

Code Bite

```
func executeTransfer(amount: Double,
  balance: Double) throws -> String {
    guard amount > 0 else {
      throw TransferError.invalidAmount
    }
    guard amount <= balance else {
      throw TransferError.insufficientFunds(
                available: balance)
    }
    return "Transfer of $\(amount) complete"
}
```

©Logical Imagination Group LLC

Handling Errors

- Invoke the function that may throw inside a do block of code
- Precede the function call with the keyword try
- The do block can be followed by any number of catch blocks that will catch specific kinds of errors

Code Bite

```
do {
    let result = try executeTransfer(
       amount: 500, balance: 1000)
    print(result)
} catch let e as TransferError {
    print(e.localizedDescription)
} catch {
    print(error)
}
```

©Logical Imagination Group LLC

Without Generics

- Without generics you have code duplication
- You must create a separate function for each data type

Code Bite

```
func first(_ arr: [Int]) -> Int? {
  arr.first
}

func first(_ arr: [String]) -> String? {
  arr.first
}

first([1, 2, 3])          // Int?
first(["a", "b"])         // String?
```

©Logical Imagination Group LLC

Generics - Write Once, Work for Any Type

- With generics, one function can work for ALL types
- The possible operations you can do with the data is limited
  - The method could apply to literally any data type
  - You don't know that the data values have any particular method or property

Code Bite

```
func first<T>(_ arr: [T]) -> T? {
  arr.first
}

first([1, 2, 3])            // Int?
first(["a", "b"])           // String?
first([Transaction]())      // Transaction?
```

©Logical Imagination Group LLC

Constrained Generics

- You can gain more knowledge of the data type involved in a generic function by limiting the range of data types to which it could apply
- A generic constraint limits the data types to those that conform to one or more protocols.
- This way you can work with some specific attributes of the data type

Code Bite

```
func findLargest<T: Comparable>(_ arr: [T])
-> T? {
    arr.max()
}

findLargest([3, 1, 7, 2])         // 7
findLargest([3.14, 2.71])         // 3.14
findLargest(["banana", "apple"]) // "banana"
```

©Logical Imagination Group LLC

Creating a Generic Struct

- Structs and classes can be generic
- The data type is specified when the instance is created
- All attributes of the struct or class can make use of the data type for parameters, variables, and return values

Code Bite

```
struct Stack<T> {
   private var items: [T] = []
   mutating func push(_ item: T) {
      items.append(item)
   }
   mutating func pop() -> T? {
      items.popLast()
   }
   var top: T? { items.last }
   var isEmpty: Bool { items.isEmpty }
}
```

©Logical Imagination Group LLC

Using a Generic Stack

- The data type is specified when creating the instance
- Separate instances could use different data types

Code Bite

```
var tasks = Stack<String>()

tasks.push("Wash the car")
tasks.push("Fold the laundry")
tasks.push("Wash the dishes")

while !tasks.isEmpty {
   print(tasks.pop()!)
}
```

©Logical Imagination Group LLC

Safety Features

Skill Check

- Find and follow the Swift code file in the GitHub repository for the Phase2/Module04/SafetyFeatures directory.

©Logical Imagination Group LLC

Block 5 Check - Safety Features

Skill Check

- Draw a retain cycle between two classes. Which reference do you make weak? Why that one?
- What is the difference between try, try?, and try!? Give a real situation for each.
- You call .map on an Optional<String>. The closure returns a String. What is the return type?
- Write a generic function signature that finds the minimum value in an array of any type. What constraint do you need?

©Logical Imagination Group LLC